

Week 08 Explained Student Worksheet

Vector Space Model and Cosine Similarity

Name _____ Student ID _____ Group _____

Represent a query in the same TF-IDF space as the documents, then rank the collection by cosine similarity. Compute every score from the text.

Use this worksheet during the two-hour practical session. Read, predict, run, and explain. The existing lab sheet contains the complete program; this companion explains key ideas and provides space to record your own evidence. Python 3.10 or later is sufficient; no pip packages are needed.

What you should be able to do

- Construct a query vector using the document vocabulary and IDF.
- Implement dot product, magnitude, and cosine similarity manually.
- Rank documents by decreasing similarity with deterministic ties.
- Handle zero vectors and out-of-vocabulary query terms.

Understand the concepts

In the vector space model, a document and a query are vectors over the same vocabulary. The query uses its own raw term counts multiplied by the IDF values learned from the document collection. Do not add the query to the collection or recalculate IDF for each search.

The dot product sums products of matching coordinates. The magnitude is the square root of the sum of squared coordinates. Cosine similarity divides the dot product by both magnitudes. It measures direction agreement rather than raw vector length. With these nonnegative weights, scores range from zero to one, allowing for tiny floating-point error.

The formula is undefined if either vector has magnitude zero. The implementation returns 0.0 by explicit convention. Empty, stop-word-only, or entirely unknown queries therefore produce no positive score. Terms absent from the vocabulary are ignored when constructing a query vector.

The teaching program prints all documents, including zero scores. Week 9 evaluates positive-score results at a fixed cutoff. Equal scores are ordered by document ID so reruns agree. A rank is a computed order, not a guarantee that a document is relevant to a human information need.

Predict before running

For $q=[1,1]$ and $d=[1,0]$, predict whether cosine is zero, one, or a value between them.

Prepare your terminal

Extract the course and open a terminal inside Week_08_Vector_Space_Model. Keep docs and data beside the Python files. The main demonstration is lab08_solution.py; your exercise starter is lab08_exercises.py. Starters run but deliberately contain unfinished TODO functions.

```
py -3.13 lab08_solution.py
py -3.13 lab08_exercises.py
```

Use py -3 or python instead if that is your working Python 3.10+ command.

Worked example with explanation

Calculate the similarity between $q = [1, 1]$ and $d = [1, 0]$ by hand. Their dot product is 1, their magnitudes are $\sqrt{2}$ and 1, and their cosine is $1/\sqrt{2}$.

```
import math
q = [1.0, 1.0]
d = [1.0, 0.0]
dot = sum(q[i] * d[i] for i in range(len(q)))
q_size = math.sqrt(sum(value * value for value in q))
d_size = math.sqrt(sum(value * value for value in d))
score = dot / (q_size * d_size)
print(f"{score:.4f}")
```

Expected output from an executed example

0.7071

How each stage produces the result

1. The dot product multiplies corresponding coordinates and adds the products. Here it is 1.
2. Each magnitude is the square root of the sum of squared coordinates. The query magnitude is $\sqrt{2}$; the document magnitude is 1.
3. Dividing by the product of magnitudes gives approximately 0.7071. It measures directional similarity rather than a count of matching words.
4. Scaling d by a positive factor does not change its direction or cosine. The complete implementation also guards zero magnitudes and rejects unequal vector lengths.

Explain one intermediate value in your own words

Read the weekly program

The following excerpt is copied exactly from lab08_solution.py. It depends on the imports, constants, and helper functions in that file. Read it there rather than pasting it into an empty script and expecting the excerpt to run alone.

```
def dot_product(v1, v2):
    if len(v1) != len(v2):
        raise ValueError("Vectors must have the same length")
    return sum(v1[i] * v2[i] for i in range(len(v1)))

def magnitude(vector):
    return math.sqrt(sum(value * value for value in vector))

def cosine_similarity(v1, v2):
    product = dot_product(v1, v2)
    denominator = magnitude(v1) * magnitude(v2)
    if denominator == 0:
        return 0.0
    return product / denominator
```

Code walkthrough

1. dot_product explicitly checks vector lengths. A silent zip over unequal lengths would omit coordinates and hide a model error.
2. magnitude uses the Euclidean length. cosine_similarity returns 0.0 if either input has zero magnitude, avoiding division by zero.
3. query_vector reuses document vocabulary order and collection IDF. Unknown query terms have no coordinate; an all-unknown or all-stop-word query can have zero weight.
4. rank_query sorts by descending score and then document ID for ties. Week 8 shows zero-score entries too; Weeks 9 and 10 retain positive scores for evaluation or display.

For a nonzero vector v , what should cosine_similarity(v , v) return? What should the cosine function return for two zero vectors?

Connection to the course

Week 9 compares ranked output with independently defined relevance judgments. Similarity provides an ordering, while relevance tells us whether that ordering serves the information need.

Guided practical activity

Work through the steps in order. Compare your predictions with observations and record at least one specific value per experiment. Keep original baseline data until you have saved the first transcript. Put experiments that change files in a disposable copy.

Step 1 Reuse the Week 7 document collection, vocabulary, and IDF calculations.

Changing preprocessing, vocabulary, or IDF in only one side makes coordinate comparisons inconsistent.

Step 2 Implement `dot_product`, `magnitude`, and `cosine_similarity` in separate functions.

Test vector operations on short hand-checkable lists before using collection vectors.

Step 3 Tokenize the query and count query term frequencies.

Preserve repeated query terms so query TF can change direction.

Step 4 Construct the query vector in the existing vocabulary order, using the collection IDF.

Do not fit a new IDF model to the query alone. Use the collection model.

Step 5 Calculate a score against every document, then sort by decreasing score and increasing ID for ties.

Check both score values and deterministic tie ordering. A rank number is not itself a relevance judgment.

Step 6 Run the default query and at least two custom queries. Test empty-weight queries and record why they have zero scores.

Compare several information needs. Record whether a zero result came from unknown words, removed stop words, or zero IDF coordinates.

Test record

Check to demonstrate	Actual observation	Pass or revise
A nonzero vector compared with itself has cosine near 1		
A zero vector produces 0 without division by zero		
Length mismatch raises ValueError		

Record one prediction that changed after running the program and explain why

Independent work and submission

Complete these selected tasks in your own exercise file. Use the existing lab sheet for additional exercises. Do not submit the unchanged starter or simply import a solution in place of implementing the requested function.

- Hand calculation: compute cosine for [1, 2] and [2, 1], then verify it with your function.
- Implementation: complete the starter and add explicit handling of zero vectors.
- Debugging: reorder one vector without reordering the other. Explain why the resulting dot product is meaningless.
- Interpretation: compare queries python data and python python data. Explain how query TF changes the direction.

Diagnose common errors

A division-by-zero error occurs. Check the magnitude product before dividing. The course convention returns 0.0 for zero-weight vectors.

Scores look plausible but refer to different terms. Compare the vocabulary labels for every coordinate; equal lengths alone do not guarantee matching semantics.

Submit these items

- Completed vector functions with normal, zero-vector, and length-mismatch tests.
- Computed rankings for three queries using the unchanged collection.
- A short explanation of query weighting, tie order, and out-of-vocabulary handling.

Your evidence should identify the input or query, expected result, actual result, and explanation. State any document edits or preprocessing changes so the result can be reproduced. Include this completed worksheet or a clearly numbered digital response.

Exit questions

Why must query and document coordinates refer to the same terms?

Why should query processing reuse collection IDF?

Optional challenge

Implement sparse vectors as dictionaries from terms to weights and compare scores with the dense baseline. Measure storage and calculation counts on the same collection.

Source for this companion

Based on the supplied Week 08 lab sheet, shipped program, exercise starter, and sample data. The complete main program and expected_output.txt in this folder are the references for checking your baseline run.